# Off-Policy GRPO for Online Speculative Decoding

*MSAI-635 Interim Report*

Gaurav Prakash UID: 122250130
Sanjana Kommareddy UID: 122099364
Simran Kharbanda UID: 122283671

## 1 Introduction and Scope

Speculative decoding lets a small *draft* model propose $K$ tokens that a large *target* model verifies in one forward pass; verification keeps the longest correct prefix plus one target token, so the output is exactly the target's. The speed-up depends almost entirely on the mean acceptance length $\bar{\tau}$ (accepted draft tokens per verification step). Our proposal argued that the drafter should be trained *online*, on the traffic it serves, and proposed replacing the usual forward-KL distillation loss with an off-policy GRPO objective that rewards accepted length directly.

**Change of plan since the proposal.** The published online-training system (Aurora) targets Qwen3-8B on several 80 GB GPUs and depends on a CUDA-only stack. We first need a forward-KL baseline that we fully control, so the project has two phases:

1. **Base-paper reproduction (nearly complete):** online EAGLE-3 draft training, scaled down to Qwen3-0.6B and run with PyTorch's MPS backend on Mac laptops. The draft model, loss and data code are reused from the paper's release; the serving and orchestration layers are our own.
2. **GRPO extension (next):** the off-policy GRPO objective from the proposal, built on the reproduction. The system already logs the quantities GRPO needs: per-round accepted length (the reward) and, for every round, the served draft version and the trainer's version (the staleness $\Delta$).

## 2 Progress So Far

The reproduction is split into nine work items, each with a concrete "done when" criterion (Table 1). Seven are complete and merged; the online system and its evaluation run end to end.

**Data.** The paper's mixed-domain training data is private, so we built public replacements: 1000 conversations each from ShareGPT (chat), CodeAlpaca-20k (code) and GSM8K (math), filtered to start with a user turn and to fit in 1024 tokens. From these pools we emit three request streams: *mixed* (3000 requests, shuffled), *ordered* (3000, grouped by domain) and *domain-shift* (1200: 600 chat, then 300 code, then 300 math). The draft predicts a reduced 24K-token vocabulary (out of Qwen3's 152K), built from assistant-token frequencies on the mixed stream.

**Models.** The target is Qwen3-0.6B (596M parameters), with auxiliary hidden states captured from layers 1, 13 and 24. The EAGLE-3 draft is a single Llama decoder layer (hidden size 1024) fed by a linear fusion of the three auxiliary states (3072→1024) and the token embedding: 42.4M trainable parameters (24.6M in its 24K-row output head) plus a frozen 155.6M-parameter embedding copied from the target. Training uses a three-step test-time-training (TTT) unroll with loss $\sum_i 0.8^i \mathcal{L}_i$, where $\mathcal{L}_i$ is the cross-entropy against the target's soft distribution at unroll step $i$.

**Static baseline.** No public EAGLE-3 draft exists for Qwen3-0.6B, so we trained one offline on the chat pool, holding out its last 100 conversations: AdamW at $10^{-4}$, warmup plus cosine, effective

**Table 1:** Reproduction work items and their status.

| # | Work | Done-when criterion | Status |
|---|---|---|---|
| 1 | Mac environment, device layer (CUDA → MPS → CPU), EAGLE-3 code installed without CUDA-only dependencies | Draft and EAGLE-3 modules import; parser and vocab tests pass | Done |
| 2 | Public datasets (chat, code, math), three request streams, 24K draft-vocab mapping | Streams and cached mapping on disk | Done |
| 3 | Qwen3-0.6B target with auxiliary hidden-state capture; EAGLE-3 draft and trainer step on MPS | Draft overfits one batch in 200 steps; time and memory measured | Done |
| 4 | Offline static EAGLE-3 baseline trained on held-in chat data | $\bar{\tau}$ clearly $> 0$ on held-out chat | Done |
| 5 | Speculative-decoding server: draft, verify, accept, KV cache, logging, training samples, weight hot-swap | Greedy output token-identical to plain decoding on 50 prompts | Done |
| 6 | Online loop: sample pool, trainer, versioned weight publish; sync and async; day-0 and with-draft starts | $\bar{\tau}$ rises over 300 day-0 requests | Done |
| 7 | Metrics, latency bench, CSV/summary writer, plots, online losslessness regression test | Plots and tables from the run logs | Done |
| 8 | Controlled experiments: domain shift, ordered stream, sync interval, learning rate, sync vs. async | Results table and figures | Next |
| 9 | Scale check on a CUDA GPU with Qwen3-1.7B/4B | Same code runs on CUDA | Optional |

batch 8, fp32 weights, gradient checkpointing, 8 epochs (896 steps, 19 minutes). The 170 MB checkpoint is published as a GitHub release with a SHA-256 checksum, so every member starts from identical weights. It is both the *static* baseline and the starting point for *with-draft* online training.

**Inference server.** An HF-transformers draft–verify loop at batch size 1. Each round the draft proposes $K = 3$ guesses and the target scores them in one pass using a `DynamicCache` (cropped on rejection), with greedy (exact argmax match) or speculative-sampling ($\min(1, p/q)$, residual resampling) acceptance. It logs per-round accepted lengths and draft versions, emits training samples, and can load new draft weights between requests.

**Online loop.** The server's training samples go into a bounded *sample pool* (128 samples, kept on the CPU); serving blocks when the pool is full rather than dropping samples. A separate *online trainer* owns its own fp32 copy of the draft and runs the static-baseline recipe on batches from the pool at a constant learning rate ($10^{-4}$ day-0, $10^{-5}$ with-draft). Every $N$ steps it *publishes* a copy of its weights into a single pending slot, versioned by its step count; the server applies it before its next request. In *sync* mode the loop alternates between serving $B$ requests and training $M$ steps. In *async* mode a serving thread and a training thread run concurrently; because MPS aborts when two threads issue GPU work at once, a FIFO device lock serialises their GPU calls. Every request records the trainer's version, so staleness $\Delta$ is exact.

**Evaluation tooling.** All metrics are recomputed from a run's request and training logs: rolling $\bar{\tau}$, per-position acceptance $\alpha_i$, acceptance rate, tokens/s against target-only decoding, the estimated speed-up $(1 + \bar{\tau})/(1 + Kc)$, staleness and the loss curve. A latency bench measures $c$, the cost of one draft step relative to one target step. Output is CSV, JSON and plots. The repository has 68 tests, including losslessness under live weight swaps in sync and async modes.

# 3 Notional Architecture

Figure 1 shows the system as built. The GRPO extension changes only the trainer's loss and adds tree drafting to the drafter, to produce groups of rollouts; everything else is reused. Compared with the paper's deployment, the multi-GPU serving engine, transfer store, cluster scheduler and sharded trainer are replaced by an in-process pool, a single-device trainer and an in-memory publish slot;

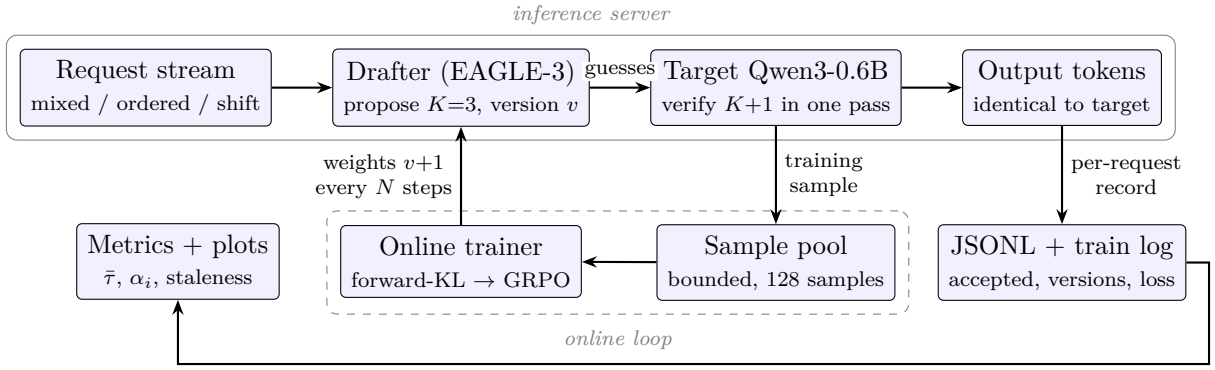



**Figure 1:** System architecture (all components implemented). The trainer's objective is forward KL in the reproduction and off-policy GRPO in the extension.

the target shrinks from 8B to 0.6B, the draft vocabulary from 32K to 24K, and $K$ and the TTT length from 5 to 3.

# 4 Frameworks and Libraries

- **PyTorch 2.14** on the MPS backend (Apple GPU), eager mode with SDPA attention; tests also run with CPU fallback disabled. Python threads for the async loop.
- **Hugging Face** `transformers` 4.57.1 (target model, `DynamicCache`, chat templates), `datasets` and `huggingface_hub` (ShareGPT, CodeAlpaca, GSM8K), `accelerate`, `safetensors`.
- **Paper's EAGLE-3 code** (installed without its CUDA-only dependencies): draft model, TTT wrapper, target-feature hooks, vocab mapping, conversation parser.
- **Tooling:** `uv` with Python 3.12, `pytest`, `matplotlib`, a Docker CPU image, a cloud CUDA notebook for an independent GPU check, GitHub releases for checkpoints and one issue / PR per work item.

# 5 Baselines and State of the Art

**Lineage.** Speculative decoding with lossless verification (Leviathan et al.; Chen et al., 2023) uses a separate small LM as the draft. Medusa adds parallel decoding heads to the target. The EAGLE family drafts from the target's own hidden states; EAGLE-2 adds dynamic draft trees, and EAGLE-3 fuses low-, mid- and high-level features and trains with a test-time-training unroll. EAGLE-3 is the state-of-the-art drafter and the one we use. Online Speculative Decoding first proposed distilling the draft on live traffic, and Aurora turns this into a unified training–serving system with asynchronous weight hot-swap; this is the method we reproduce. GRPO is the critic-free RL objective our extension adapts.

**Systems we compare.** All are evaluated on the same streams and decoding settings: (1) **target only**, the speed-up reference; (2) **static EAGLE-3**, the frozen offline draft ("train offline, then deploy"); (3) **online day-0**, online forward-KL training from a random draft; (4) **online with-draft**, online forward-KL training from the static draft; (5) **ours**, off-policy GRPO with a forward-KL anchor, with the ablations from the proposal (synchronous on-policy GRPO, $\Delta = 0$; group size $G$; KL weight $\beta$; clipping and truncation thresholds; reward shaping).

**Planned experiments.** Day-0 online training against target-only on the mixed stream (done, below); static vs. with-draft under the chat→code→math domain shift, measured as the area under the rolling-$\bar{\tau}$ curve after each shift; the same on the ordered stream; and sensitivity to the sync interval $N \in \{5, 20, 50\}$, the learning rate, and sync vs. async training. The headline metric is $\bar{\tau}$,

**Table 2:** Main results for each draft model.

| Draft model | Starts from | Online | Requests | $\bar{\tau}$ | Speed vs. target | Lossless |
|---|---|---|---|---|---|---|
| Untrained (sanity check) | random | no | 50 mixed | 0.00 | 0.56× | 50/50 |
| Overfit (upper bound) | memorised | no | 50 mixed | 2.60 | 2.0× | 50/50 |
| **Static baseline** | offline training | no | 100 held-out | **0.31** | 0.59× | 100/100 |
| **Online day-0**, whole run | random | yes | 300 mixed | 0.15 | 0.53× | 300/300 |
| last 50 requests | | | | **0.25** | – | |
| Online with-draft (test) | static baseline | yes | 20 mixed | ≈0.26 | – | – |

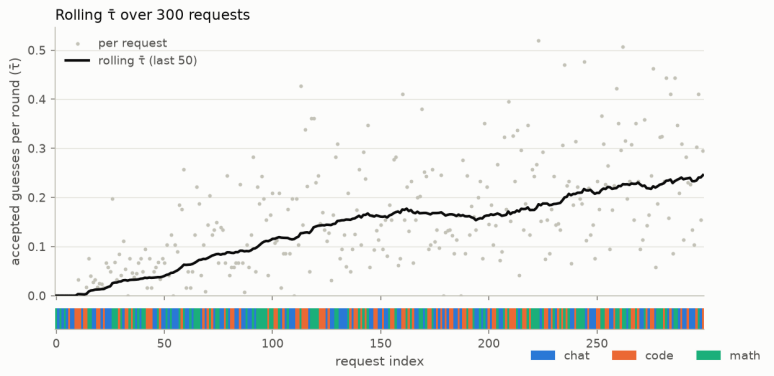



**Figure 2:** Online day-0 run: $\bar{\tau}$ per request (dots) and averaged over the last 50 requests (line), with each request's domain underneath.

which does not depend on hardware; we also report $\alpha_i$, measured and estimated speed-up, and the staleness distribution.

# 6 Evaluation Results (Prototype)

All runs use MPS on Mac laptops, greedy decoding, $K = 3$ guesses per round and up to 128 new tokens. We report two numbers per run: $\bar{\tau}$, the average number of guesses accepted per round (higher is better, at most 3), and the measured speed relative to decoding with the target alone ($> 1\times$ means faster). "Lossless" counts requests whose output was token-identical to the target alone.

**Static baseline.** The draft trained offline for 8 epochs guesses the target's next token correctly 31% of the time on held-out chat, and accepts $\bar{\tau} = 0.31$ guesses per round when served.

**Online day-0.** Starting from a random draft and learning only from the 300 requests it served, $\bar{\tau}$ climbs from 0.04 over the first 50 requests to **0.25** over the last 50 (Figure 2), and is still rising. It learns fastest on math (0.23 over the run), then code (0.14), then chat (0.09). The weights were swapped 30 times during the run and every output stayed identical to the target's.

**Online with-draft.** Starting from the static baseline, the draft accepts ≈0.26 guesses per round from the first request, a level day-0 only reaches after 300 requests. A full run is the next step.

**What the numbers say.**

1. **Speculative decoding here is always lossless**: 450 of 450 checked requests matched the target exactly, including during live weight swaps.
2. **Online learning works from scratch**: with only 150 training steps, day-0 reaches 80% of the static draft, which needed 896 offline steps.
3. **Nearly all accepted guesses are the first guess.** Later guesses build on the draft's own

earlier guesses, so errors compound. This is where a reward for accepted length (GRPO) should help most.

4. **No speed-up yet.** A round costs about 2.2 target steps but yields only $1 + \bar{\tau}$ tokens, so a speed-up needs $\bar{\tau} \gtrsim 1.2$. The overfit row shows the server does give 2× once guesses are good.

**Relation to published results.** Papers report tokens produced per target pass ($1 + \bar{\tau}$): about 4–6 for EAGLE-2/3 with tree drafting on 8B–70B targets, giving 3–6× speed-ups. We are at about 1.3 and below 1×. The gap is expected: our drafts have seen thousands rather than hundreds of thousands of conversations, we draft one chain of 3 instead of a tree, a 0.6B target makes the draft relatively expensive, and a laptop at batch size 1 lacks the kernels and batching of production servers. We therefore compare our systems against each other on the same setup, with $\bar{\tau}$ as the headline metric.

# 7 Issues Faced

**Solved.**

- **Compute and CUDA-only stack:** scaled down to Qwen3-0.6B on MPS with our own server, pool, trainer and publish slot, while reusing the published draft, loss and data code unchanged.
- **No public draft, private data:** trained our own EAGLE-3 draft on public data; ShareGPT filters drop 939 threads that start mid-conversation and 613 longer than 1024 tokens.
- **Draft-vocab coverage:** 16K covered only 97.6% of assistant tokens (others can never be drafted); 24K covers 0.993 / 0.995 / 0.998 (chat / code / math) for 8.2M more parameters.
- **Memory:** 4×1K-token batches needed 9.9 GB (draft logits × 3 TTT steps); batch 2 with gradient checkpointing trains in 2.4 GB, and the whole online loop peaks at 7.5 GB, so it fits a 16 GB laptop.
- **Precision:** pure-bf16 AdamW lost 2–3 accuracy points (small updates round away), so the trainer always keeps fp32 weights and checkpoints reload in their trained dtype. In bf16, a 4-token verify pass rounds differently from 4 single steps and can flip near-ties, so losslessness checks run in fp32 (450/450 pass).
- **Threads on MPS:** MPS aborts when two threads use the GPU at once; async mode serialises GPU calls with a FIFO device lock.
- **Sharing checkpoints:** the static draft is a GitHub release with a SHA-256 checksum.

**Open.**

- **No wall-clock speed-up yet.** Levers: higher $\bar{\tau}$ (more online training, GRPO), a draft KV cache, and tree drafting.
- **Train/test overlap in the streams.** The 600 chat requests of the domain-shift stream, and 900 of the 1000 chat requests in the mixed stream, were static-baseline training prompts. This does not affect day-0 runs, but before the domain-shift comparisons we must rebuild the chat segments from held-out conversations or report on unseen requests only.
- **Single-device asynchrony.** With the device lock, "async" on one GPU interleaves rather than overlaps serving and training, so staleness is real but smaller than with separate serving and training GPUs.
- **Hardware variance.** A fanless laptop slowed ∼12% over an hour (the speed ratio held at 0.53–0.54), so we only compare speeds within a run.
- **GRPO prerequisites.** Groups of rollouts need tree drafting (top-$k > 1$), which is not yet implemented, and $K = 3$ gives a shorter reward range than $K = 5$.

## 8 Next Steps

1. **With-draft run:** 300 mixed requests from the released static draft, with the same settings as the day-0 run, for a direct comparison.
2. **Controlled experiments**, after removing the stream overlap: static vs. day-0 vs. with-draft on the domain-shift and ordered streams, and sweeps over $N$, learning rate and sync vs. async.
3. **GRPO extension:** tree drafting for groups; reward $R(a) = L(a) - \lambda\kappa(K - L(a))$ from the logged accepted lengths; the off-policy objective with truncated importance weights and the forward-KL anchor, using the logged staleness; and comparison against systems 2–4 above.
4. **Scale check (optional):** the same code on a CUDA GPU with a larger target, to check that the trends carry over.